SALUS SECURITY

MAY 2026

CODE SECURITY ASSESSMENT

Four

Overview

Project Summary

- Name: Openfour
- Platform: BSC Blockchain
- Language: Solidity
- Repository:
    - https://github.com/dev-four-d4/openfour
- Audit Range: See Appendix - 1

Project Dashboard

Application Summary

Name	Openfour
Version	v6
Type	Solidity
Dates	May 29 2026
Logs	May 07 2026, May 11 2026, May 12 2026, May 15 2026, May 25 2026, May 29 2026

Vulnerability Summary

Total High-Severity issues	10
Total Medium-Severity issues	13
Total Low-Severity issues	7
Total informational issues	5
Total	35

Contact

E-mail: support@salusec.io

Risk Level Description

High Risk	The issue puts a large number of users' sensitive information at risk, or is reasonably likely to lead to catastrophic impact for clients' reputations or serious financial implications for clients and users.
Medium Risk	The issue puts a subset of users' sensitive information at risk, would be detrimental to the client's reputation if exploited, or is reasonably likely to lead to a moderate financial impact.
Low Risk	The risk is relatively small and could not be exploited on a recurring basis, or is a risk that the client has indicated is low impact in view of the client's business circumstances.
Informational	The issue does not pose an immediate risk, but is relevant to security best practices or defense in depth.

Content

Introduction **4**

1.1 About SALUS 4

1.2 Audit Breakdown 4

1.3 Disclaimer 5

Findings **6**

2.1 Summary of Findings 6

2.2 Notable Findings 8

1. FeeRouter assets may be drained through a fake OpenFour token 8
2. Migration may be blocked by donation 9
3. Missing slippage protection in swapForQuote 10
4. Fee rewards may be incorrectly distributed during pool buys 11
5. Incorrect created pool address calculation 12
6. Possible sqrtPriceX96 overflow 13
7. Missing token0/token1 check 14
8. V4 migration skips anti-sniper buyback-burn 15
9. Hook initialization may be manipulated 16
10. Users can add liquidity into the pool before migrate 17
11. The setCustomData function cannot work as expected 18
12. The transferOwnership function cannot work as expected 19
13. Blacklisted users can bypass fee-claim restrictions by transferring tokens 20
14. Assets may be stuck in the token helper 21
15. Missing decimal conversion in _tryGetMarketCap 22
16. Token price may be manipulated 23
17. Bypass tax payment via deploying a new pool 24
18. Missing signature verification 25
19. Incorrect presale calculation 26
20. Possible incorrect slippage protection 27
21. Possible unfair tax fee distribution 28
22. Users can earn profit via the higher pool price 29
23. Centralization risk 30
24. The withdrawEmergence() is unreachable from the Core contract 31
25. Incorrect initial B0/T0 calculation for pancakeswapV2 32
26. Fee distribution may block buy/sell operations 33
27. Missing validation in upsertPreset 34
28. Incorrect rounding direction for buy/sell 35
29. Implementation contract could be initialized by everyone 36
30. Possible reentrancy attack 37

2.3 Informational Findings 38

31. LikwidV2Migrate module does not work on BSC mainnet 38
32. Suggest revert with the specific error msg 39
33. V4 migration deadline computed at execution time 40
34. Unnecessary initialize variable 41
35. Unnecessary hook address check 42
Appendix 1 - Files in Scope 43

Introduction

1.1 About SALUS

At Salus Security, we are in the business of trust.

We are dedicated to tackling the toughest security challenges facing the industry today. By building foundational trust in technology and infrastructure through security, we help clients to lead their respective industries and unlock their full Web3 potential.

Our team of security experts employ industry-leading proof-of-concept (PoC) methodology for demonstrating smart contract vulnerabilities, coupled with advanced red teaming capabilities and a stereoscopic vulnerability detection service, to deliver comprehensive security assessments that allow clients to stay ahead of the curve.

In addition to smart contract audits and red teaming, our Rapid Detection Service for smart contracts aims to make security accessible to all. This high calibre, yet cost-efficient, security tool has been designed to support a wide range of business needs including investment due diligence, security and code quality assessments, and code optimisation.

We are reachable on Telegram (https://t.me/salusec), Twitter (https://twitter.com/salus_sec), or Email (support@salusec.io).

1.2 Audit Breakdown

The objective was to evaluate the repository for security-related issues, code quality, and adherence to specifications and best practices. Possible issues we looked for included (but are not limited to):

- Risky external calls
- Integer overflow/underflow
- Transaction-ordering dependence
- Timestamp dependence
- Access control
- Call stack limits and mishandled exceptions
- Number rounding errors
- Centralization of power
- Logical oversights and denial of service
- Business logic specification

- Code clones, functionality duplication

1.3 Disclaimer

Note that this security audit is not designed to replace functional tests required before any software release and does not give any warranties on finding all possible security issues with the given smart contract(s) or blockchain software, i.e., the evaluation result does not guarantee the nonexistence of any further findings of security issues.

Findings

2.1 Summary of Findings

ID	Title	Severity	Category	Status
1	FeeRouter assets may be drained through a fake OpenFour token	High	Business Logic	Resolved
2	Migration may be blocked by donation	High	Business Logic	Resolved
3	Missing slippage protection in swapForQuote	High	Business Logic	Resolved
4	Fee rewards may be incorrectly distributed during pool buys	High	Business Logic	Acknowledged
5	Incorrect created pool address calculation	High	Business Logic	Resolved
6	Possible sqrtPriceX96 overflow	High	Business Logic	Resolved
7	Missing token0/token1 check	High	Business Logic	Resolved
8	V4 migration skips anti-sniper buyback-burn	High	Business Logic	Resolved
9	Hook initialization may be manipulated	High	Business Logic	Resolved
10	Users can add liquidity into the pool before migrate	High	Business Logic	Resolved
11	The setCustomData function cannot work as expected	Medium	Business Logic	Resolved
12	The transferOwnership function cannot work as expected	Medium	Business Logic	Resolved
13	Blacklisted users can bypass fee-claim restrictions by transferring tokens	Medium	Business Logic	Resolved
14	Assets may be stuck in the token helper	Medium	Business Logic	Resolved
15	Missing decimal conversion in _tryGetMarketCap	Medium	Business Logic	Resolved
16	Token price may be manipulated	Medium	Business Logic	Resolved
17	Bypass tax payment via deploying one new pool	Medium	Business Logic	Acknowledged
18	Missing signature verification	Medium	Business Logic	Resolved

19	Incorrect presale calculation	Medium	Business Logic	Resolved
20	Possible incorrect slippage protection	Medium	Business Logic	Acknowledged
21	Possible unfair tax fee distribution	Medium	Business Logic	Resolved
22	Users can earn profit via the higher pool price	Medium	Business Logic	Acknowledged
23	Centralization risk	Medium	Centralization	Acknowledged
24	The withdrawEmergence() is unreachable from the Core contract	Low	Business Logic	Acknowledged
25	Incorrect initial B0/T0 calculation for pancakeswapV2	Low	Business Logic	Resolved
26	Fee distribution may block buy/sell operations	Low	Business Logic	Acknowledged
27	Missing validation in upsertPreset	Low	Business Logic	Acknowledged
28	Incorrect rounding direction for buy/sell	Low	Business Logic	Acknowledged
29	Implementation contract could be initialized by everyone	Low	Business Logic	Resolved
30	Possible reentrancy attack	Low	Business Logic	Acknowledged
31	LikwidV2Migrate module does not work on BSC mainnet	Informational	Business Logic	Resolved
32	Suggest revert with the specific error msg	Informational	Business Logic	Resolved
33	V4 migration deadline computed at execution time	Informational	Business Logic	Resolved
34	Unnecessary initialize variable	Informational	Business Logic	Resolved
35	Unnecessary hook address check	Informational	Business Logic	Resolved

2.2 Notable Findings

Significant flaws that impact system confidentiality, integrity, or availability are listed below.

1. FeeRouter assets may be drained through a fake OpenFour token

Severity: High	Category: Business Logic
Target: - contracts/core/OpenFourFeeRouter.sol	

Description

The function distributeFeesFromBlanace authorizes callers via `onlyCoreOrTokenMigrateModule(token)`. But the `token` parameter is fully caller-controlled. The modifier trusts `IOpenFourToken(token).migrateModule()` without verifying that `token` is a real OpenFour token registered in OpenFourCore.

An attacker can deploy a malicious token contract whose migrateModule function returns the attacker's address. The attacker can then call `distributeFeesFromBalance` with this fake token and transfer any unreserved ERC20 balance held by the FeeRouter.

contracts/core/OpenFourFeeRouter.sol:L224-L248

```
function distributeFeesFromBalance(address token, address quoteAsset, uint256
evalTotalFee, FeeRecipient[] memory recipients)
    external
    onlyCoreOrTokenMigrateModule(token)
{
    ...
    uint256 totalDistributed;
    for (uint256 i = 0; i < recipients.length; i++) {
        FeeRecipient memory recipientVo = recipients[i];
        uint256 amount = recipientVo.amount;
        if (amount == 0) continue;
        totalDistributed += amount;
        IERC20(quoteAsset).safeTransfer(recipientVo.recipient, amount);
        emit FeeAssigned(token, quoteAsset, recipientVo.recipient, recipientVo.kind,
amount, recipientVo.pending);
    }
    require(totalDistributed == evalTotalFee, "FeeRouter: total fee mismatch");
}
```

Recommendation

Validate the input token contract.

Status

This issue has been resolved by the team with commit 7332063.

2. Migration may be blocked by donation

Severity: High	Category: Business Logic

Target:
- contracts/libraries/PancakeV2MigrateLib.sol

Description

When the sold token reaches the target, the remaining meme token will be added into the pancake swap pool via `addLiquidity`.

Before the migration, users are not allowed to transfer meme tokens into the migrated pools. In most cases, the pool will be empty before this migration.

The problem here is that users can donate quote tokens to increase the reserve amount in the pancake pool via `donate/sync`. When the migration contract wants to add liquidity, this operation will be reverted.

contracts/libraries/PancakeV2MigrateLib.sol:L54-L90

```
    function executePancakeLiquidity(
        LiqParams memory p,
        OpenFourTypes.MigrateHookContext calldata ctx
    ) internal {
        (uint256 usedToken, uint256 usedQuote,) =
IPancakeV2RouterLike(p.router).addLiquidity(
            ctx.token,
            ctx.quoteAsset,
            p.tokenLiquidityAmount,
            p.quoteToUse,
            p.amountTokenMin,
            p.amountQuoteMin,
            recipient,
            block.timestamp + p.deadlineOffset
        );
}
```

Recommendation

Consider using pair.mint directly.

Status

This issue has been resolved by the team with commit ecce423.

3. Missing slippage protection in swapForQuote

Severity: High	Category: Business Logic
Target: - contracts/globals/TokenHelper.sol	

Description

In TaxToken, the accumulated fee will be swapped to quote tokens and distribute these fees to holders, founders, etc.

The problem here is that there is no swap slippage protection here. Malicious users can trigger one MEV attack on this.

contracts/globals/TokenHelper.sol:L96-L112

```
function _swapForQuote(address token, address quote, uint256 amountToken) internal
returns (uint256) {
    address[] memory path = new address[](2);
    path[0] = token;
    path[1] = quote;

    IERC20(token).forceApprove(pancakeRouter, amountToken);
    uint256[] memory amounts =
IPancakeRouterLike(pancakeRouter).swapExactTokensForTokens(
        amountToken,
        0,
        path,
        address(this),
        block.timestamp
    );
    // amount0 is input token. Amounts 1 is the output token amount.
    return amounts[1];
}
```

Recommendation

Calculate the expected output amount via the TWAP price. If the actual output token amount is far away from the expected output amount, revert this transaction.

Status

This issue has been resolved by the team with commit 60bb0ab.

4. Fee rewards may be incorrectly distributed during pool buys

Severity: High	Category: Business Logic

Target:
- contracts/token/TaxToken.sol

Description

In TaxToken, the accumulated fee will be swapped to quote tokens and distribute these fees to holders, founders, etc.

When users buy meme tokens from the pool, some fees will be accumulated. But the function `_dispatchFee` is not triggered. This will cause that this fee per share is not updated. Users may lose the fees because of the staled fee per share.

contracts/token/TaxToken.sol:L140-L187

```
function _transfer(address from, address to, uint256 amount) internal override {
    require(to != address(this), "TaxToken: invalid recipient");
    if (tokenPhase == OpenFourTypes.Phase.Migrated && !swapping) {
        if (toPool) {
            _dispatchFee();
        }
    if (tokenPhase == OpenFourTypes.Phase.Migrated) {
        _updateShare(from);
        if (from != to) {
            _updateShare(to);
        }
        if (!swapping && !fromPool && !toPool) {
            _dispatchFee();
            _claimFee(from);
        }
    }
}
```

Recommendation

When users buy meme tokens from the pool, dispatch the fee.

Status

This issue has been acknowledged by the team.

5. Incorrect created pool address calculation

Severity: High	Category: Business Logic

Target:
- contracts/library/PancakePoolDiscoveryLib.sol

Description

In PancakeSwapV2MigrateModule, all related pools for meme/quote pairs will be calculated and set as the migrated pools.

The problem here is that the actual contract for the new pair's deployment is not the factory contract, but the poolDeployer contract.

contracts/library/PancakePoolDiscoveryLib.sol:L154-L172

```
    function _discoverV3Pool(address factory, address token, address quoteAsset, uint24
fee)
        private
        view
        returns (address pool, bool alreadyExists)
    {
        try IPancakeV3FactoryLike(factory).getPool(token, quoteAsset, fee) returns
(address p) {
            pool = p;
        } catch {
            return (address(0), false);
        }
        alreadyExists = pool != address(0);
        // Even if it does not exist, we can compute this address.
        if (!alreadyExists) {
            (address token0, address token1) = _sortTokens(token, quoteAsset);
            bytes32 salt = keccak256(abi.encode(token0, token1, fee));
            pool = _computeCreate2Address(factory, salt, V3_POOL_INIT_CODE_HASH);
        }
    }
```

Recommendation

Calculate the pair's address via the pool deployer, not the factory.

Status

This issue has been resolved by the team with commit 7d91484.

6. Possible sqrtPriceX96 overflow

Severity: High	Category: Business Logic

Target:
- contracts/libraries/PancakeV4MigrateLib.sol

Description

The function `_computeSqrtPriceX96` computes a sqrt ratio for `poolManager.initialize()`. Any presale raising more than ~18.4 units of an 18-decimal quote token triggers this bug.

At PancakeV4MigrateLib.sol, `uint256 ratio = (num << 192) / denom` left-shifts by 192 bits. When `num > 2^64`, the shift overflows silently in Solidity 0.8+, producing a corrupted `sqrtPriceX96`.

The V4 pool initializes at a wrong price. LP funds are deposited at incorrect ratios and permanently trapped, or the migration reverts entirely.

contracts/libraries/PancakeV4MigrateLib.sol:L185-L198

```
function _computeSqrtPriceX96(uint256 quoteAmount, uint256 tokenAmount, bool
tokenIsCurrency0)
    private pure returns (uint160)
{
    require(quoteAmount > 0 && tokenAmount > 0, "V4MigrateLib: zero amounts");
    uint256 num   = tokenIsCurrency0 ? quoteAmount : tokenAmount;
    uint256 denom = tokenIsCurrency0 ? tokenAmount : quoteAmount;
    uint256 ratio = (num << 192) / denom;
    return uint160(_sqrt(ratio));
}
```

Recommendation

Use safe math to avoid the possible overflow.

Status

This issue has been resolved by the team with commit 170f67c.

7. Missing token0/token1 check

Severity: High	Category: Business Logic
Target: - contracts/libraries/PancakeV4MigrateLib.sol	

Description

The contract calculates liquidity for the V4 pool and feeds it to `positionManager.modifyLiquidities`. The bug triggers whenever token address > quote address lexicographically.

At PancakeV4MigrateLib.sol:216-230, `_computeLiquidity` assumes token is always currency0 and quote is currency1. But `addFullRangeLiquidity` sorts both via `CurrencyLibrary.sort()`. When token > quoteAsset, the roles invert but the formulas stay the same.

Only a fraction of funds enter the pool. The remainder — most of the deposited value — sits permanently trapped in the migrate module with no recovery path.

contracts/libraries/PancakeV4MigrateLib.sol:L216-L230

```
function _computeLiquidity(
    uint256 quoteAmount, uint256 tokenAmount, uint160 sqrtPriceX96, int24, int24
) private pure returns (uint128) {
    uint256 Q96 = 2 ** 96;
    uint256 liqFromToken = (uint256(sqrtPriceX96) * tokenAmount) / Q96;
    uint256 liqFromQuote = (quoteAmount * Q96) / uint256(sqrtPriceX96);
    uint256 liq = liqFromToken < liqFromQuote ? liqFromToken : liqFromQuote;
    require(liq > 0 && liq <= type(uint128).max, "V4MigrateLib: liquidity OOB");
    return uint128(liq);
}
```

Recommendation

Check if the token is token0 or token1 and then calculate the liquidity.

Status

This issue has been resolved by the team with commit 7ec9f1e.

8. V4 migration skips anti-sniper buyback-burn

Severity: High	Category: Business Logic

Target:
- contracts/modules/migrate/PancakeSwapV4MigrateModule.sol

Description

During the token's trading phase, buyers who trigger the anti-sniper mechanism pay an additional fee that accumulates in the vault as `antiSniperQuoteAccrued`. After migration, these accumulated fees should be swapped for tokens and burned — the V2 migration modules (`PancakeSwapV2MigrateModule`, `BondingLikwidV2MigrateModule`) both execute this step after adding liquidity.

`executeMigration` calls `addFullRangeLiquidity(lp)` to deposit LP, but never calls `_executeAntiBuybackBurn`. There is no mechanism after migration to recover the accumulated anti-sniper fees.

`antiSniperQuoteAccrued` is permanently locked in the vault. The amount depends on trading volume during the token's lifetime and can be substantial.

contracts/modules/migrate/PancakeSwapV4MigrateModule.sol:L156-L187

```
function executeMigration(OpenFourTypes.MigrateHookContext calldata ctx, bytes calldata)
    external override onlyFourCore
    returns (uint8 migratedDataVersion, bytes memory encodedMigratedData)
{
    // ...
    bytes32 poolId = PancakeV4MigrateLib.addFullRangeLiquidity(lp);
    // missing: _executeAntiBuybackBurn(ctx, poolId);
    migratedDataVersion = V4_MIGRATED_DATA_V1;
    encodedMigratedData = abi.encode(poolId);
}
```

Recommendation

Process the accrued anti sniper fees properly.

Status

This issue has been resolved by the team with commit f86348b.

9. Hook initialization may be manipulated

Severity: High	Category: Business Logic
Target: - contracts/modules/migrate/PancakeSwapV4MigrateModule.sol	

Description

The V4 migration calls `poolManager.initialize` to create the pool at the computed sqrt price. Initialize is only callable once per pool key. The pool key is determined by token and quote addresses and is publicly computable from token creation.

At PancakeV4MigrateLib.sol:96, `p.poolManager.initialize(key, sqrtPriceX96)` is called without checking whether the pool already exists. The pool key is static and predictable — anyone can compute it and call `initialize` at any time during the trading phase (hours or days before migration).

If an attacker initializes the pool first with a different price, the migration's `initialize` reverts. Since initialization is one-time per pool, the migration path is permanently blocked for that token with no recovery.

contracts/modules/migrate/PancakeSwapV4MigrateModule.sol:L156-L187

```
function addFullRangeLiquidity(LiqParams memory p) internal returns (bytes32 poolId) {
    // ...
    (Currency c0, Currency c1) = CurrencyLibrary.sort(p.token, p.quoteAsset);
    // ...
    PoolKey memory key = PoolKey({
        currency0: c0, currency1: c1, hooks: hooksAddr,
        poolManager: address(p.poolManager), fee: p.fee,
        parameters: PancakeInfinityClParams.encodePoolParameters(p.tickSpacing,
hooksBitmap)
    });
    poolId = bytes32(PoolId.unwrap(key.toId()));
    uint160 sqrtPriceX96 = _computeSqrtPriceX96(p.quoteToUse, p.tokenLiquidityAmount,
p.token < p.quoteAsset);
    p.poolManager.initialize(key, sqrtPriceX96);
    // ...
}
```

Recommendation

Initialize this hook when the system creates token or block other users initialize this hook via hook.

Status

This issue has been resolved by the team with commit a7afc2e.

10. Users can add liquidity into the pool before migrate

Severity: High	Category: Business Logic
Target: - contracts/modules/migrate/PancakeSwapV4MigrateModule.sol	

Description

During initialization, the module registers V4 infrastructure addresses as "`migrated pools`" to block token transfers to them during Trading. This prevents premature LP interactions before migration. However, it registers the wrong addresses.

At PancakeSwapV4MigrateModule.sol, `poolManager` and `positionManager` are registered via `setMigratedPool`. In Pancake V4 Infinity, token transfers during `modifyLiquidity` and `swap` do not go to the PoolManager or PositionManager contracts. Instead, the PoolManager's internal `_pay` method pulls tokens via Permit2 to `address(vault)` — the PoolManager's internal vault. Only this vault address receives token transfers, yet it is not registered.

If an attacker pre-initializes the pool, they can add liquidity to it during Trading. The ERC20 token transfer goes to the PoolManager's internal vault, which is not blocked by migratedPools, so `_beforeTokenTransfer` passes. The `poolManager` and `positionManager` addresses themselves never receive token transfers, so registering them has no protective effect.

contracts/modules/migrate/PancakeSwapV4MigrateModule.sol:L73-L129

```
function init(address token, address fourCore_, address feeRouter_, bytes calldata
rawParams, string calldata moduleVersion_)
    external override initializer
{
    // ...
    IUniTokenModule tm = IUniTokenModule(tokenModuleAddr);
    address pm_  = tm.poolManager();
    address pos_ = tm.positionManager();
    // ...
    IOpenFourToken(token).setMigratedPool(pm_, true);
    IOpenFourToken(token).setMigratedPool(pos_, true);
    // ...
}
```

Recommendation

Process the accrued anti sniper fees properly.

Status

This issue has been resolved by the team with commit f86348b.

11. The setCustomData function cannot work as expected

Severity: Medium	Category: Business Logic
Target: - contracts/token/OpenFourToken.sol	

Description

When a preset declares a non-zero `customDataId`, the deployer's `executeCreate` creates a customData BeaconProxy and calls `token.setCustomData(customDataInst)` to register it on the token.

`OpenFourToken.setCustomData` has the `onlyFourCore` modifier which requires `msg.sender == vault.fourCore()`. The vault's `fourCore` was set to the Core address during vault initialization. However, `setCustomData` is called from the Deployer contract — `msg.sender` is the Deployer, not the Core. The check always fails and the call always reverts.

`token.customData` stays `address(0)` permanently. The `setCustomData` function has `require(customData == address(0))`, so the first (and only) attempt already failed. External integrators reading `IOpenFourToken(token).customData()` get zero. The Core's internal mapping stores the correct customData address, so internal protocol functions work, but the token's own state is desynchronized and cannot be corrected after deployment.

contracts/modules/migrate/OpenFourToken.sol:L73-L129

```
modifier onlyFourCore() {
    require(msg.sender == IOpenFourVault(vault).fourCore(), "Token: only fourCore");
    _;
}
function setCustomData(address cd) external onlyFourCore {
    require(customData == address(0), "Token: customData already set");
    customData = cd;
}
```

Recommendation

Process the accrued anti sniper fees properly.

Status

This issue has been resolved by the team with commit dc64bbf.

12. The transferOwnership function cannot work as expected

Severity: Medium	Category: Business Logic
Target: - contracts/modules/token/UniTokenModule.sol	

Description

After the UniTokenV4Hook is deployed via CREATE2, the constructor sets `owner = msg.sender` (the UniTokenModule). `hook.setToken(token)` binds the token address but `hook.transferOwnership(token)` is never called. The UniTokenModule remains the permanent owner.

The hook has `setToken` and `transferOwnership` guarded by `onlyOwner`. Since the UniTokenModule has no code path to call these functions after init, they are permanently inaccessible — dead code.

contracts/modules/token/UniTokenModule.sol:L69-L129

```
function createToken(
    address creator, address token, address vault, address curveModule,
    address tradeModule, address migrateModule,
    OpenFourTypes.TokenCreateParams calldata tokenParams,
    string calldata tokenImplVersion, string calldata tokenModuleVersion
) external override returns (uint256 maxSupply, string memory name, string memory
symbol) {
    // ...
    UniTokenV4Hook hook = _deployMinedHook(IPoolManager(v4.poolManager));
    _deployedHook = address(hook);
    hook.setToken(token);
    // ...
}
```

Recommendation

Add missing interfaces.

Status

This issue has been resolved by the team with commit 5ea1daa.

13. Blacklisted users can bypass fee-claim restrictions by transferring tokens

Severity: Medium	Category: Business Logic
Target: - contracts/token/TaxToken.sol	

Description

In TaxToken, one user can be added into the blacklist by the shareholder manager. After that, users cannot claim fees.

The problem here is that users in the blacklist can still transfer tokens to another address to bypass this limitation.

contracts/token/TaxToken.sol:L363-L394

```
function _claimFee(address account) internal {
    if (quote == address(0)) {
        return;
    }
    if (shareHolderManager != address(0) &&
IShareHolderManager(shareHolderManager).isBlacklisted(account)) {
        return;
    }
}
```

Recommendation

Block token transfer for users in the blacklist.

Status

This issue has been resolved by the team with commit 5d55750.

14. Assets may be stuck in the token helper

Severity: Medium	Category: Business Logic
Target: - contracts/globals/TokenHelper.sol	

Description

In TokenHelper contract, the `addLibrary` function will swap 50% meme token to quote token, and then add meme/quote tokens into the pancake pair.

The problem here is that there may be some dust after the liquidity is added. These remaining tokens will be stuck in this helper contract.

contracts/globals/TokenHelper.sol:L165-L190

```
function addLiquidity(uint256 amountToken) external {
    address token = msg.sender;
    address quote = ITokenLike(token).quote();
    require(quote != address(0), "TokenHelper: zero quote");

    _receiveToken(token, amountToken);
    // Some asset will be locked here.
    uint256 amountSwap = amountToken / 2;
    uint256 amountQuote = _swapForQuote(token, quote, amountSwap);
    uint256 amountTokenRemain = amountToken - amountSwap;

    IERC20(token).forceApprove(pancakeRouter, amountTokenRemain);
    IERC20(quote).forceApprove(pancakeRouter, amountQuote);
    IPancakeRouterLike(pancakeRouter).addLiquidity(
        token,
        quote,
        amountTokenRemain,
        amountQuote,
        0, // min amount is 0.
        0,
        address(0xdEaD), // dead address.
        block.timestamp
    );
}
```

Recommendation

Add an interface to withdraw remaining tokens.

Status

This issue has been resolved by the team with commit 48d65c0.

15. Missing decimal conversion in _tryGetMarketCap

Severity: Medium	Category: Business Logic

Target:
- contracts/globals/CreatorFeeManager.sol

Description

In CreatorFeeManager, the function `_tryGetTokenPrice` will be used to calculate token's price and current usd cap.

The problem here is that the function does not consider the decimal conversion and uses `reserve0/reserve1` amount directly.

For example, 1e18 DAI and 1e6 USDT's usd value should be the same. But according to the calculation below, the calculation result will be scaled with the decimal difference.

contracts/globals/CreatorFeeManager.sol:L292-L312

```
function _tryGetTokenPrice(address token, address quote) internal view returns (bool ok,
uint256 price) {
    if (token == quote) return (true, 1e18);
    if (token == address(0)) token = wrappedNative;
    if (quote == address(0)) quote = wrappedNative;
    if (token == address(0) || quote == address(0) || token == quote) return (false, 0);
    // This pair is from one uniswap v2 pool.
    address pair = IUniswapV2FactoryLike(dexFactory).getPair(token, quote);
    if (pair == address(0)) return (false, 0);
    // get reserve amount. reserve0 and reserve1.
    (uint112 r0, uint112 r1,) = IUniswapV2PairLike(pair).getReserves();
    if (r0 == 0 || r1 == 0) return (false, 0);
    // token0 & token1
    address token0 = IUniswapV2PairLike(pair).token0();
    address token1 = IUniswapV2PairLike(pair).token1();
    if (token == token0 && quote == token1) {
        // token1 amount * 1e18 / token0 amount.
        return (true, Math.mulDiv(uint256(r1), 1e18, uint256(r0)));
    }
    if (token == token1 && quote == token0) {
        return (true, Math.mulDiv(uint256(r0), 1e18, uint256(r1)));
    }
    return (false, 0);
}
```

Recommendation

Consider different tokens' decimals when calculating a token's price.

Status

This issue has been resolved by the team with commit 1f1e943.

16. Token price may be manipulated

Severity: Medium	Category: Business Logic

Target:
- contracts/globals/CreatorFeeManager.sol

Description

In CreatorFeeManager, the function `_tryGetTokenPrice` will be used to calculate token's price and current usd cap.

The function uses the `reserve0/reserve1` amount directly. The problem here is that the reserve0/reserve1 amount can be manipulated via swap. This may lead to an incorrect price tier than expected.

contracts/globals/CreatorFeeManager.sol:L292-L312

```
    function _tryGetTokenPrice(address token, address quote) internal view returns (bool
ok, uint256 price) {
        if (token == quote) return (true, 1e18);
        if (token == address(0)) token = wrappedNative;
        if (quote == address(0)) quote = wrappedNative;
        if (token == address(0) || quote == address(0) || token == quote) return (false,
0);
        // This pair is from one uniswap v2 pool.
        address pair = IUniswapV2FactoryLike(dexFactory).getPair(token, quote);
        if (pair == address(0)) return (false, 0);
        // get reserve amount. reserve0 and reserve1.
        (uint112 r0, uint112 r1,) = IUniswapV2PairLike(pair).getReserves();
        if (r0 == 0 || r1 == 0) return (false, 0);
        // token0 & token1
        address token0 = IUniswapV2PairLike(pair).token0();
        address token1 = IUniswapV2PairLike(pair).token1();
        // @audit-ok Medium missing decimal conversions.
        // @audit-issue Medium this price can be manipulated.
        if (token == token0 && quote == token1) {
            // token1 amount * 1e18 / token0 amount.
            return (true, Math.mulDiv(uint256(r1), 1e18, uint256(r0)));
        }
        if (token == token1 && quote == token0) {
            return (true, Math.mulDiv(uint256(r0), 1e18, uint256(r1)));
        }
        return (false, 0);
    }
```

Recommendation

Suggest using twap price.

Status

This issue has been resolved by the team with commit d7f9d5d.

17. Bypass tax payment via deploying a new pool

Severity: Medium	Category: Business Logic
Target: - contracts/token/TaxToken.sol	

Description

In TaxToken, users will pay some tax when they buy/sell meme tokens from the migrated pools.

The problem here is that users can deploy another pool to bypass this tax fee. For example, the original pool is the Meme/USDC pool. Users can deploy a Meme/USDT pool to bypass paying tax.

contracts/token/TaxToken.sol:L140-L187

```
function _transfer(address from, address to, uint256 amount) internal override {
    require(to != address(this), "TaxToken: invalid recipient");

    bool fromPool = migratedPools[from];
    bool toPool = migratedPools[to];
    if (tokenPhase == OpenFourTypes.Phase.Migrated && !swapping) {
        if (toPool) {
            _dispatchFee();
        }
        bool isBuy = fromPool && to != vault;
        bool isSell = toPool && from != vault;
        if (isBuy || isSell) {
            uint256 feeRate = isBuy ? buyFeeRate : sellFeeRate;
            uint256 fee = (amount * feeRate) / 10_000;
            if (fee > 0) {
                amount -= fee;
                super._transfer(from, address(this), fee);
                feeAccumulated += fee;
                totalTaxCollected += fee;
            }
        }
    }
}
```

Recommendation

Charge tax fee if users buy/sell meme tokens via any pancake pool.

Status

This issue has been acknowledged by the team.

18. Missing signature verification

Severity: Medium	Category: Business Logic
Target: - contracts/core/OpenFourCore.sol	

Description

When users create one meme token via the function `createToken`, the signer will sign this signature with the defined creator fee. The meme token creator should pay the defined creator fee.

The problem here is that the signature verification is missing. This will cause users to bypass paying creator fees.

contracts/core/OpenFourCore.sol:L79-L98

```
function createToken(bytes memory createArgs, bytes memory signature)
    external
    payable // pay native token for create fee.
    whenNotPaused
    nonReentrant
    returns (address token)
{
    // The caller is in the process of creating a token, so we can verify the signature.
Nobody else can trigger this one.
    // _verifyCreateTokenSignature(_msgSender(), createArgs, signature);

    (address tokenAddr, address vaultAddr, uint256 presaleQuote, uint256 nativeFp) =
OpenFourCoreLib.executeCreateToken(
        usedRequestIds,
        tokens,
        version,
        wrappedNative,
        address(registry),
        address(feeRouter),
        createArgs
    );
}
```

Recommendation

Add the signature verification process.

Status

This issue has been resolved by the team with commit aa3f561.

19. Incorrect presale calculation

Severity: Medium	Category: Business Logic
Target: - contracts/core/OpenFourCore.sol	

Description

When the antiSniper feature is enabled, users need to pay extra sniper fees for this trade based on quote amount. The start block's antiSniper fee is high, which will reach the quote amount. It means that if users can buy 100 meme tokens with 100 quote tokens according to current curve calculation, users need to pay around 200 quote tokens because of the existing antiSniper fees.

The problem here is that in the presale phase, the antiSniper fees will be calculated based on the `presaleQuote`. The antiSniper fee amount will be around `presaleQuote`. Then the calculated `curveBudge` will be one small value, and it's possible to be zero. This will cause the presale to not work as expected.

contracts/core/OpenFourCore.sol:L119-L179

```
function _executePresale(
    address token,
    IOpenFourVault vault,
    uint256 presaleQuote,
    uint256 nativeForPresale
) internal {
        ...
        (uint256 estimatedFee,,) =
            feeRouter.previewDistribution(presaleQuote, tradeEst.fees,
cfg.antiSniperEnabled, cfg.createBlock);
        uint256 budgetDeduction = estimatedFee;
        // Here the curve budget will be deducted via .
        curveBudget = presaleQuote > budgetDeduction ? presaleQuote - budgetDeduction :
0;
}
```

Recommendation

Follow the normal buy calculator and refactor the presale calculation.

Status

This issue has been resolved by the team with commit aa3f561.

20. Possible incorrect slippage protection

Severity: Medium	Category: Business Logic
Target: - contracts/libraries/OpenFourCoreLib.sol	

Description

When users buy a meme token via the function `buy`, users can provide one `maxQuotePayAmount` as the slippage protection. If the current price is higher than expected, this trade will be reverted.

The problem here is that users' expected bought amount might be adjusted. If users' bought amount is adjusted and becomes less than expected value, it's improper to use the original `maxQuotePayAmount` as the slippage protection.

contracts/libraries/OpenFourCoreLib.sol:L79-L98

```
function processBuyPayment(
    address wrappedNative,
    address quoteAsset,
    address vault,
    address feeRouter,
    address token,
    uint256 presetId,
    uint256 curveQuote,
    uint256 maxQuotePayAmount,
    OpenFourTypes.FeeTier[] memory fees,
    uint256 nativeValue,
    bool antiSniperEnabled,
    uint256 createBlock
) external returns (TradePaymentResult memory r) {
    IOpenFourFeeRouter.FeeRecipient[] memory recipients;
    (recipients,, r.totalFee, r.protocolFee, r.taxFee, r.devFee, r.antiSniperFee) =
IOpenFourFeeRouter(feeRouter)
        .buildDistribution(token, presetId, curveQuote, fees, quoteAsset,
antiSniperEnabled, createBlock);
    r.traderQuote = curveQuote + r.totalFee + r.antiSniperFee;
    // slippage protection.
    // @audit-issue Medium here the maxQuotePayAmount
    if (r.traderQuote > maxQuotePayAmount) revert IOpenFourErrors.CoreErrSlippage();
    ...
}
```

Recommendation

Adjust the `maxQuotePayAmount` according to the actual bought amount.

Status

This issue has been acknowledged by the team.

21. Possible unfair tax fee distribution

Severity: Medium	Category: Business Logic
Target: - contracts/token/TaxToken.sol	

Description

In TaxToken, the tax fees will be distributed to holders, founders, dead addresses, etc. If the swap fails, one part of fees will not be deducted from the `feeAccumulated`. These remaining fees will be distributed in the next round.

The problem here is that the burn part will always succeed. This will cause that in the next round, the system will re-calculate fee distribution, holders will lose some expected fees.

contracts/token/TaxToken.sol:L241-L346

```
    function _dispatchFee() internal {
        if (holderActive) {
            amountHolder = (amountTotal * rateHolder) / rateTotal;
            if (amountHolder > 0) {
                (bool holderOk, uint256 holderQuote, bytes memory holderReason) =
_trySwapForQuote(address(this), amountHolder);
                if (holderOk) {
                    amountHolderDone = amountHolder;
                    feeHolder += amountHolder;
                    amountDispatched += amountHolder;
                    dispatchedQuoteHolder = holderQuote;
                    if (holderQuote > 0) {
                        feePerShare += (holderQuote * MAGNITUDE) / totalShares;
                        quoteHolder += holderQuote;
                    }
                } else {
                    emit FeeDispatchDeferred(2, amountHolder, holderReason);
                }
            }
        }
}
```

Recommendation

Make sure that all parts of fee distribution succeed or fail together.

Status

This issue has been resolved by the team with commit cc1c3d2.

22. Users can earn profit via the higher pool price

Severity: Medium	Category: Business Logic
Target: - contracts/libraries/PancakeV2MigrateLib.sol	

Description

When one token is mitigated, the collected antiSniper fees will buy back some meme token. This operation will increase the meme token's price in the pool.

In the initial calculation, the expected behavior is that the latest price before mitigation will be similar with the initial price in the pool. The problem here is that the buy back behavior will cause the meme token's price to increase in this pool. The last buyer can buy meme tokens via openfour and then sell meme tokens via the mitigated pool to earn profits.

contracts/libraries/PancakeV2MigrateLib.sol:L94-L139

```
function executeBuybackBurn(
    address router,
    address vault,
    address quoteAsset,
    address token,
    uint256 quoteAmount,
    address burnRecipient
) internal returns (bool swapped) {
    try IPancakeV2RouterSwapLike(router).swapExactTokensForTokens(
        quoteAmount, 0, path, burnRecipient, block.timestamp
    ) {
        swapped = true;
    } catch {
        swapped = false;
    }
}
```

Recommendation

Consider minting more meme tokens and adding liquidity to dead addresses.

Status

This issue has been acknowledged by the team.

23. Centralization risk

Severity: Medium	Category: Centralization
Target: - contracts/global/TokenHelper.sol	

Description

In the OperatorManager/Ledger contracts, there exists some privileged roles, for example, `Owner`. These roles have the authority to execute some key functions such as `withdraw`.

If these roles' private keys are compromised, an attacker could trigger these functions to withdraw all funds.

contracts/global/TokenHelper.sol:L193-L199

```
    /// @notice Emergency/manual withdrawal for fallback credits.
    function withdraw(address token, address account, address to) external onlyOwner {
        uint256 amount = balances[token][account];
        require(amount > 0, "TokenHelper: no balance");
        balances[token][account] = 0;
        IERC20(token).safeTransfer(to, amount);
        emit Withdrawal(token, account, to, amount);
    }
```

Recommendation

We recommend transferring privileged accounts to multi-sig accounts with timelock governors for enhanced security. This ensures that no single person has full control over the accounts and that any changes must be authorized by multiple parties.

Status

This issue has been acknowledged by the team.

24. The withdrawEmergence() is unreachable from the Core contract

Severity: Medium	Category: Business Logic
Target: - contracts/vault/StandardVault.sol	

Description

In StandardVault, the function `withdrawEmergence` is used to withdraw assets when it's in emergency mode.

This function can only be used by the core contract. The problem here is that the core contract does not have an interface to trigger this function.

contracts/vault/StandardVault.sol:L125-L137

```
function withdrawEmergence(address to, address asset) external override onlyFourCore {
    require(to != address(0), "Vault: zero recipient");
    if (asset == address(0)) {
        uint256 bal = address(this).balance;
        if (bal == 0) return;
        (bool ok,) = payable(to).call{value: bal}("");
        require(ok, "Vault: native transfer failed");
        return;
    }
    uint256 erc20Bal = IERC20(asset).balanceOf(address(this));
    if (erc20Bal == 0) return;
    SafeERC20.safeTransfer(IERC20(asset), to, erc20Bal);
}
```

Recommendation

Add an interface in the FourCore contract to support this.

Status

This issue has been acknowledged by the team.

25. Incorrect initial B0/T0 calculation for pancakeswapV2

Severity: Medium	Category: Business Logic

Target:
- contracts/module/curve/BondingCurveModule.sol

Description

In BondingCurveModule, the function `calcInitailLiquidity` will calculate the initial B0 and T0 value for this curve.

In this calculation, the migrateFee will be deducted from the net raised asset. The problem here is that in the contract PancakeSwapV2MigrateModule, the migration fee is 0.

contracts/module/curve/BondingCurveModule.sol:L229-L252

```
    function calcInitialLiquidity(
        uint256 maxRaising_,
        uint256 totalSupply_,
        uint256 offers_,
        uint256 migrateFeeBps
    ) public pure returns (uint256 initialB0, uint256 initialT0) {
        // offers > 0
        require(offers_ > 0, "BondingCurve: offers");
        // total supply >= offers.
        require(totalSupply_ >= offers_, "BondingCurve: invalid supply");
        require(migrateFeeBps < MAX_BPS, "BondingCurve: bad migrate fee");
        uint256 reserve = totalSupply_ - offers_;
        // Here the reserve must be larger than 0. And the offers > reserve.
        require(reserve > 0 && offers_ > reserve, "BondingCurve: invalid sale ratio");
        // quote token.
        uint256 netRaiseForLiquidity = Math.mulDiv(maxRaising_, MAX_BPS - migrateFeeBps,
MAX_BPS);
        uint256 denom = Math.mulDiv(netRaiseForLiquidity, offers_, reserve) -
maxRaising_;
        require(denom > 0, "BondingCurve: invalid migrate fee");
        initialT0 = Math.mulDiv(netRaiseForLiquidity, Math.mulDiv(offers_, offers_,
reserve), denom);
        initialB0 = Math.mulDiv(maxRaising_, maxRaising_, denom);
        if (initialB0 == 0) initialB0 = 1;
    }
```

Recommendation

Calculate the B0/T0 based on the actual migrate fee bps from different migration modules.

Status

This issue has been resolved by the team with commit a9e4ca1.

26. Fee distribution may block buy/sell operations

Severity: Low	Category: Business Logic
Target: - contracts/core/OpenFourFeeRouter.sol	

Description

When users buy/sell meme tokens in the pre-sale phase, users need to pay some fees. And these fees will be distributed via the function `distributeFeesFromBalance`.

The problem here is that if a recipient is in the blacklist, this will cause the buy/sell process to be blocked.

contracts/core/OpenFourFeeRouter.sol:L224-L248

```
    function distributeFeesFromBalance(address token, address quoteAsset, uint256
evalTotalFee, FeeRecipient[] memory recipients)
        external
        onlyCoreOrTokenMigrateModule(token)
    {
        uint256 balance = IERC20(quoteAsset).balanceOf(address(this));
        uint256 pendingReserved = pendingDevFeeTotalByQuote[quoteAsset];
        require(balance >= pendingReserved + evalTotalFee, "FeeRouter: pending reserve
insufficient");

        uint256 totalDistributed;
        for (uint256 i = 0; i < recipients.length; i++) {
            FeeRecipient memory recipientVo = recipients[i];
            // pending fees leave to claim
            if (recipientVo.pending == true) continue;

            uint256 amount = recipientVo.amount;
            if (amount == 0) continue;

            totalDistributed += amount;
            IERC20(quoteAsset).safeTransfer(recipientVo.recipient, amount);
            emit FeeAssigned(token, quoteAsset, recipientVo.recipient, recipientVo.kind,
amount, recipientVo.pending);
        }
        require(totalDistributed == evalTotalFee, "FeeRouter: total fee mismatch");
    }
```

Recommendation

Use try/catch for the safeTransfer.

Status

This issue has been acknowledged by the team.

27. Missing validation in upsertPreset

Severity: Low	Category: Business Logic
Target: - contracts/core/OpenFourRegistry.sol	

Description

The operator can set or update the preset with valid module ids. The problem here is that if one module is set to in-active, the operator may set or update this module into the preset incorrectly.

contracts/core/contracts/core/OpenFourRegistry.sol:L160-L184

```
function upsertPreset(OpenFourTypes.Preset calldata preset) external onlyOperator {
    require(preset.id != 0, "Registry: empty preset");
    _requireModule(preset.tokenModuleId, ModuleKind.Token);
    _requireModule(preset.vaultModuleId, ModuleKind.Vault);
    _requireModule(preset.curveModuleId, ModuleKind.Curve);
    _requireModule(preset.tradeModuleId, ModuleKind.Trade);
    _requireModule(preset.migrateModuleId, ModuleKind.Migrate);
    OpenFourTypes.Preset storage current = presets[preset.id];
    if (current.id == 0) {
        presetIds.push(preset.id);
    }
}
```

Recommendation

Add input validation to make sure that this module is active.

Status

This issue has been acknowledged by the team.

28. Incorrect rounding direction for buy/sell

Severity: Low	Category: Business Logic

Target:
- contracts/modules/curve/BondingCurveModule.sol

Description

The function `calcBuyCostByLiquidity` is used to calculate the quote token amount when users buy a meme token.

The problem here is that the `nextB` is calculated via the rounding down. This may cause that the later K will be a little bit less than the previous K.

contracts/modules/curve/BondingCurveModule.sol:L144-L149

```
function calcBuyCostByLiquidity(uint256 k, uint256 B, uint256 T, uint256 amount) public
pure returns (uint256) {
    require(T > 0 && amount > 0 && amount < T, "BondingCurve: invalid buy");

    uint256 nextB = k / (T - amount);
    if (nextB <= B) return 0;
    return nextB - B;
    // K = (T - amount) * (B + input)
    // input = K / (T - amount) - B
}
```

Recommendation

Choose the proper rounding direction in the buy/sell process.

Status

This issue has been acknowledged by the team.

29. Implementation contract could be initialized by everyone

Severity: Low	Category: Business Logic
Target: - contracts/modules/trade/SimpleTradeModule .sol	

Description

According to OpenZeppelin, the implementation contract should not be left uninitialized.

An uninitialized implementation contract can be taken over by an attacker, which may impact the proxy. There is nothing preventing the attacker from calling the init function in SimpleTradeModule's implementation contract.

contracts/modules/trade/SimpleTradeModule.sol:L19-L27

```
contract SimpleTradeModule is IOpenFourTradeModule, IOpenFourModuleSchema {
   function init(address token, address fourCore_, bytes calldata params) external
override {
        require(!initialized, "Trade: already initialized");
        require(fourCore_ != address(0), "Trade: zero fourCore");
        require(params.length == 0, "Trade: params not empty");
        _initParams = params;
        boundToken = token;
        fourCore = fourCore_;
        initialized = true;
    }
}
```

Recommendation

To prevent the implementation contract from being used, consider invoking the _disableInitializers function in the constructor of the SimpleTradeModule contract to automatically lock it when it is deployed.

Status

This issue has been resolved by the team with commit ee868df.

30. Possible reentrancy attack

Severity: Low	Category: Business Logic
Target: - contracts/globals/TokenHelper.sol	

Description

When the contract transfers native Ether to users, the swapping flag is set to true. Users can buy/sell meme tokens with the pool without tax fees.

contracts/globals/TokenHelper.sol:L114-L144

```
function _swapForETH(address token, address to, uint256 amountToken) internal returns
(uint256) {
    address[] memory path = new address[](2);
    path[0] = token;
    path[1] = wrappedNative;
    // approve pancake router for amountToken.
    IERC20(token).forceApprove(pancakeRouter, amountToken);
    try IPancakeRouterLike(pancakeRouter).swapExactTokensForETH(
        amountToken,
        0,
        path,
        to,
        block.timestamp
    ) returns (uint256[] memory amounts) {
        uint256 cached = balances[wrappedNative][to];
        if (cached > 0) {
            balances[wrappedNative][to] = 0;
            IWrappedNativeLike(wrappedNative).withdraw(cached);
            Address.sendValue(payable(to), cached);
            emit Withdrawal(wrappedNative, to, to, cached);
        }
        return amounts[1];
    }
}
```

Recommendation

Add re-entrancy protection.

Status

This issue has been acknowledged by the team.

2.3 Informational Findings

31. LikwidV2Migrate module does not work on BSC mainnet

Severity: Informational	Category: Business Logic
Target: - contracts/libraries/LikwidV2MigrateLib.sol	

Description

The LikwidMigrate module does not set a valid config for the BSC mainnet. This will cause the LikwidMigrate module to not work on BSC mainnet.

contracts/libraries/LikwidV2MigrateLib.sol:L68-L144

```
    function configByChainId(uint256 chainId) internal pure returns (LikwidConfig memory
cfg) {
        if (chainId == BSC_TESTNET_CHAIN_ID) {
            cfg = LikwidConfig({
                likwidVault: 0x8cDbd5262db84F81d549942388D1CF08eD3BD313,
                pairPosition: 0x288ccD004eb847FC318Af262Ca57b69ABD739EA8,
                helper: 0xAb38B712fe1B13eA35F32eF623Ac5c44668498D6,
                fee: 3000,
                marginFee: 3000
            });
        }
    }
```

Recommendation

Add a proper config for BSC mainnet.

Status

This issue has been resolved by the team with commit 8778873.

32. Suggest revert with the specific error msg

Severity: Informational	Category: Business Logic
Target: - contracts/core/OpenFourCore.sol	

Description

When a trade is disallowed by the trade module, the trade module will return the specific reason. Suggest to revert with this specific reason.

contracts/core/OpenFourCore.sol:L155-L225

```
function buy(address token, uint256 amount, uint256 maxQuotePayAmount, bytes memory
proof) external payable whenNotPaused nonReentrant {
    OpenFourTypes.RuntimeConfig memory cfg = _getRuntime(token);
    ...
    OpenFourTypes.TradeResult memory tradeOut;
    // In this simple trade module, the trade hook data is empty. However, it's allowed
that the trade module returns one hook data.
    (tradeOut, tradeHookData) = IOpenFourTradeModule(cfg.tradeModule).evaluate(
         OpenFourTypes.TradeContext({
             token: token, trader: msg.sender, amount: effectiveAmount, isBuy: true,
             timestamp: block.timestamp, blockNumber: block.number,
             remainingForSale: beforeRemainingForSale, totalRaised: beforeTotalRaised,
curveQuote: curveQuote,
             proof: proof // There is one proof. It means that the trade module can
verify this proof.
         })
       );
     // Suggest to indicate the specific reason via the error message.
     if (!tradeOut.allowed) revert CoreErrTradeRejected();
     ...
}
```

Recommendation

Suggest to revert with this specific reason.

Status

This issue has been resolved by the team with commit 6c34365.

33. V4 migration deadline computed at execution time

Severity: Informational	Category: Business Logic
Target: - contracts/libraries/PancakeV4MigrateLib.sol	

Description

The `modifyLiquidities` call accepts a `deadline` parameter intended to prevent the transaction from being executed after a certain time. The passed value is `block.timestamp + 1 hours`.

`block.timestamp` is the timestamp of the block in which the transaction is mined, set at execution time. Adding 1 hour to it produces a deadline that is always 1 hour in the future when the transaction executes. If the transaction sits in the mempool for 6 hours, the deadline at execution is still `block.timestamp + 1 hours` — it never expires.

The deadline provides no protection against delayed execution. A stale migration could execute with outdated prices or state.

contracts/libraries/PancakeV4MigrateLib.sol:L68-L132

```
function addFullRangeLiquidity(LiqParams memory p) internal returns (bytes32 poolId) {
    // ...
    p.positionManager.modifyLiquidities(payload, block.timestamp + 1 hours);
    // ...
}
```

Recommendation

Use the current timestamp as the deadline.

Status

This issue has been resolved by the team with commit a36ba05.

34. Unnecessary initialize variable

Severity: Informational	Category: Business Logic
Target: - contracts/modules/migrate/PancakeSwapV4MigrateModule.sol	

Description

The `init` function checks `require(!initialized)` and sets `initialized = true`, duplicating the protection already provided by OpenZeppelin's `initializer` modifier.

The contract inherits `Initializable` and calls `_disableInitializers()` in the constructor. The `initializer` modifier on `init` already prevents re-initialization at the storage slot level. The manual `bool initialized` state variable and its associated checks are redundant, adding unnecessary storage reads, writes, and code.

contracts/modules/migrate/PancakeSwapV4MigrateModule.sol:L73-L84

```
constructor() {
    _disableInitializers();
}

function init(...) external override initializer {
    require(!initialized, "V4Migrate: already initialized");  // redundant
    // ...
    initialized = true;  // redundant
}
```

Recommendation

Remove the unnecessary `initialized` variable.

Status

This issue has been resolved by the team with commit dedba67.

35. Unnecessary hook address check

Severity: Informational	Category: Business Logic

Target:
- contracts/libraries/PancakeV4MigrateLib.sol

Description

`_resolvePoolHook` checks that the hook address's low 14 bits match `UNITOKEN_HOOK_FLAGS`. In Pancake V4 Infinity, hook callbacks are determined solely by `getHooksRegistrationBitmap()`. The address bits are never inspected by the protocol. Both the address bit `require` in `_resolvePoolHook` and the CREATE2 mining loop in `_deployMinedHook` are Uniswap V4 legacy patterns that serve no purpose here.

The `getHooksRegistrationBitmap()` check is the only validation needed. The address bit check adds gas cost and code complexity with no protective value.

contracts/libraries/PancakeV4MigrateLib.sol:L135-L142

```
function _resolvePoolHook(address h) private view returns (address hooksAddr, uint16
hooksBitmap) {
    require(h != address(0), "V4MigrateLib: zero hook");
    require((uint160(h) & ALL_HOOK_MASK) == UNITOKEN_HOOK_FLAGS, "V4MigrateLib: hook
addr flags");  // redundant
    hooksBitmap = IPancakeHooksRegistration(h).getHooksRegistrationBitmap();
    require(hooksBitmap == PancakeInfinityHookBitmap.UNITOKEN_HOOK_BITMAP,
"V4MigrateLib: hook bitmap");
    return (h, hooksBitmap);
}
```

Recommendation

Remove the unnecessary logic for the hook bit check.

Status

This issue has been resolved by the team with commit 5750d4a.

Appendix

Appendix 1 - Files in Scope

This audit covered the following files in commit b9f12c3:

File	SHA-1 hash
CreatorRewardsToken .sol	9e4a6e96fb33ad4569c1bb0d7eac597936391e0a
TaxToken.sol	0431a15979061fbbfd3312f41d4952329092cec3
OpenFourToken.sol	b06211bb1cef7950ae659dd5830eaf5b6763b443
TokenHelper.sol	966427fec6dbd211f3da22fd8dca983c4911f6ab
ShareHolderManager.sol	0b66e7448c763801c15cba12e7d33406b646c5a3
CreatorFeeManager.sol	59aeab6d6016e25d36e9c2452fb2097b1b8ff084
OpenFourFeeRouter.sol	76881b2db0c35070d4b7d82ffafa417232d7cb19
OpenFourCore.sol	729398f5abc43bd59fec8687c89e6301d7d27665
OpenFourRegistry.sol	62dbb94fa71b21b7307293b821128d3eda481112
OpenFourTools.sol	dd492052570b8c202dc611897660f6c472469f44
OpenFourDeployer.sol	1628e72488dd1a03c798438c2d9549bfd91c0e28
PancakeV2MigrateLib.sol	a829422440ea501db7cef83dced1a83f915e6ce2
LikwidV2MigrateLib.sol	4feab181e5bd114aa8857575c4d6049484c9716c
PancakePoolDiscoveryLib.sol	0b0d6bef3dd5fcda040ede9b3aed5b2afb500e22
OpenFourTypes.sol	d00e9e7320576d7e74d138010234ad06ef3f1fc9
OpenFourCoreLib.sol	ddd17b18500983cd4bf68ee0a2798109d84a194f
SignatureVerifier.sol	bd7eba39579484f542f5f14d673c8299f31dd39b
OpenFourBasicPresetValidator.sol	de0cbf96bc2a5958adc62800f42957900ee0f46f
OpenFourPresetParamValidator.sol	8daba2f81bc2b47c012ed4416030cd172f5f2502
CreatorRewardsTokenModule.sol	6484d07d0c8a972a1b51c720d3a29069de2e1b3f
TaxTokenModule.sol	cf7ef4bf1bbc9b4eb071fc99519ac5795e6ac8d5
StandardTokenModule.sol	fda95caabeac0d08d0299162974f9f0c44665154
SimpleTradeModule.sol	7058b73e11881695f7432b70864bf4b5c546127f

BondingCurveModule.sol	ef042e232859d963a4c725bfdef30b1ff727a04e
BondingLikwidV2MigrateModule.sol	1844f0ed67ffde330d40f29e94ab3eb0ca5e297f
PancakeSwapV2MigrateModule.sol	9aa0f08d7ce775090fa3032d1e7116f4bc0c40d3
BondingPcsV2MigrateModule.sol	f3156270ee574073db35aa3e0007c67fae5d47f6
UseSignerUpgradeable.sol	c559b125baf9df151730f40937f0cce3570f5529
BasicAccessControlUpgradeable.sol	043c34fa34454132ebced40b6fc7dbc4fc9a8cca
StandardVault.sol	4a9f7a08d4ba0d3b2bb9b9c35759d010435cd9db
IOpenFourCore.sol	9242cfcf49dab0f7af75cf57daf8636cc6a101fe
IOpenFourPresetValidator.sol	093149558570acb36a36d028cb9406b12469e22d
IOpenFourDeployer.sol	932bd0346f0fa485aad802be76603814e900fac7
IOpenFourCurveModule.sol	f13b0449b360e87a29eb16440081a4e61cd06f1e
IOpenFourFeeRouter.sol	5f2fc5e6b37eb6ef863178c924ef3b3dda67f41d
IOpenFourMigrateModule.sol	8f1af784fb5a15823a9b7c1fb2a8ca6adf246df6
IOpenFourToken.sol	0fb56873eaf2a447ab4735a032eb8693b23b7d4e
IOpenFourVault.sol	2b2a86c4f73e2a6cc3f662bd128717662e99cb5e
IOpenFourTradeModule.sol	199f57495219210cd33a2c3a566c6358c5f14a80
IOpenFourTokenModule.sol	16cb85e3495046561596b671edf1c91ba1a54ac0
IOpenFourRegistry.sol	48193fcf4b80f1761b98237f32ccecb4ccc5b9aa
IOpenFourTools.sol	c3802335c67dac41beaf74555c0daded5383f800
IOpenFourModuleSchema.sol	00b7750dfd04a2f0bb1ecd892927ecf0e429a9c9

And this audit covered the final versions in commit af9ac4a.